



SAPIENZA
UNIVERSITÀ DI ROMA

Intelligenza Artificiale

Sapienza Università di Roma

Facoltà di Ing. dell'Informazione, Informatica e Statistica

Laurea in Informatica

Prof. Toni Mancini

<http://tmancini.di.uniroma1.it>

Progetto studenti B.1.1 (PS.B.1.1)

Parte B

Agenti Basati su Conoscenza in Logica del
Primo Ordine

La Tecnica del Magic Set

Versione 2014-05-08

1

Introduzione

Questo documento illustra brevemente la tecnica del Magic Set per la concatenazione in avanti efficiente in Logica del Primo Ordine.

Il documento è stato redatto dagli studenti Simone Maurizi ed Igor Radomirov come parte del loro esame di Intelligenza Artificiale, e viene pubblicato senza alcuna garanzia, da parte del docente, circa la sua correttezza e completezza.



SAPIENZA
UNIVERSITÀ DI ROMA

Magic Set

Progetto di Intelligenza Artificiale a.a. 2012 - 2013

Simone Maurizi 1422441 simone.maurizi@gmail.com

Igor Radomirov 1250695 igor.radomirov@gmail.com

"Any sufficiently advanced technology is

indistinguishable from magic"

-Arthur C Clarke , "Profiles of the future"

Abstract

Presentiamo i due metodi di inferenza logica: Forward Chaining e Backward Chaining, ambientati nella logica del primo ordine. Inoltre presentiamo la tecnica del Magic Set, per l'ottimizzazione di query, come strumento di inferenza logica per tagliare i fatti irrilevanti nel corso della stessa (inferenza logica). Infine mettiamo a confronto le tre tecniche e illustriamo i vantaggi concreti dell'ottimizzazione ottenuti con il Magic Set.

INDICE

Introduzione	4
1. Datalog.....	5
2. La logica del primo ordine.....	6
2.1 Regole Horn	8
2.2 EDB Extensional Database	10
2.3 IDB Intentional Database	10
2.4 Modus Ponens	11
3. Concatenazione all'indietro	12
4. Concatenazione in avanti.....	15
5. Magic Set	19
5.1 Binding pattern.....	19
5.1.1 L'algoritmo Binding-Pattern(KB,Q)	19
5.1.2 Esempio	23
5.2 Insiemi Magici	24
5.3 Trasformazione magica.....	25
5.3.1 Passo 1: Magic facts.....	25
5.3.2 Passo 2: Magic rules	25
5.3.3 Passo 3: Modified rules	27
5.3.4. Esempio: Trasformazione Magica.....	27
6. Conclusioni.....	30

Introduzione

La tecnica del Magic Set risolve il problema dei dati superflui generati, deduzioni logiche corrette ma irrilevanti, nel corso di una inferenza logica. Data una KB (base di conoscenza di regole e fatti) e una query Q l'algoritmo del Magic Set, fa la quantità esatta di ragionamenti necessari e fornisce la risposta alla domanda "Sapendo che KB è giusto concludere Q?".

Noi umani abbiamo la capacità di accorgerci quando iniziamo a perdere di vista l'obiettivo nei nostri ragionamenti. Cominciamo dalle conoscenze che abbiamo acquisito e le combiniamo facendo due e due quattro, per poi arrivare ad una conclusione. Sfortunatamente, questa capacità di concentrazione è il punto debole negli algoritmi di intelligenza artificiale e allo stesso tempo il punto di forza del Magic Set.

Ad esempio vi è mai capitato, uscendo di casa di soffermarvi alla porta e chiedervi se vi servirà l'ombrello oggi oppure no, rimanendo perplessi ed immobili per più di qualche minuto? Uno cerca di ricordarsi se ha visto le previsioni meteo odierne, il suo subconscio fornisce dati della frequenza di pioggia, ultimamente si cerca di capire se da quello che sappiamo ci aspettiamo che piovga. Tuttavia le informazioni che usiamo in quell'istante sono una piccola parte di tutta la conoscenza che possiede il nostro cervello e di certo non avremmo mai osato usarla tutta per risponderci a tale quesito. Poiché per gli algoritmi di intelligenza artificiale la risposta potrebbe trovarsi sempre dietro l'angolo e non hanno il nostro senso di concentrazione, essi fanno esattamente questo. Prendiamo ad esempio la concatenazione in avanti, Forward Chaining (FC), avrebbe fatto tutte le conclusioni possibili per poi verificare che tra questi ci sarebbe anche "Sì, devo prendere l'ombrello." Oppure "No, oggi è una bella giornata.". Questo comporterebbe la generazione di una massa enorme di informazioni del tutto scorrelati dall'obiettivo, tra cui per esempio la somma di tutti i numeri che sappiamo. Se siamo inclini a pensare che il problema è la partenza, perché cominciamo dalle basi e cerchiamo l'obiettivo è sufficiente guardare la concatenazione all'indietro, Backward Chaining(BC) per osservare lo stesso difetto. Questa si chiederebbe "come mai mi è venuto il dubbio dell'ombrello?" e, scomponendo il suo obiettivo cercherebbe di arrivare a certezze che sappiamo essere giuste. Peccato che questa procedura non saprebbe distinguere se il fatto di avere dubbi sull'ombrello proviene dal fatto che abbiamo un ombrello o che ci aspettiamo che piovga. Nel primo caso andrebbe a chiedersi perché abbiamo un ombrello e giungerebbe alla conclusione che l'abbiamo comprato nel negozio, fatto vero ma completamente irrilevante.

<< Che quantità di informazione generano gli algoritmi in preda alla distrazione? >>

<< Potenzialmente infinita. >>

<< E quanta ne possiamo eliminare? >>

<< Grazie al Magic Set, tutta! >>



1. Datalog

Datalog è un linguaggio di programmazione logica dichiarativo, ideato durante un workshop negli anni 70-80, che riunì diversi esperti di inferenza logica e di database deduttivi come Hervé Gallaire e Jack Minker. I database deduttivi nascono dalla necessità di estendere il potere espressivo dei linguaggi di interrogazione per basi di dati strutturate secondo il modello relazionale. Lo sviluppo di Bancilhon (1986), della tecnica degli insiemi magici (Magic Set) per la riscrittura di regole, permise poi alla concatenazione in avanti di "prendere in prestito" da quella all'indietro il vantaggio di essere guidata dall'obiettivo. In favore di quest'ultima, entrarono in gioco i metodi di programmazione logica con tabelle.^[1]

Datalog è composto da:

- Simboli di predicato - indicano elementi presenti nelle tuple di una relazione della base di dati

Sintassi: `nomePredicato(attributo_1, ..., attributo_n)`

Es. `Americano(West)` significa che nella relazione di nome `Americano` c'è un elemento di nome `West`

- Regole - sono di tipo if then. Ogni regola ha un atomo di testa e zero o più atomi del corpo. Si può inferire la testa se tutti gli atomi del corpo sono soddisfatti. Gli atomi del corpo sono in congiunzione tra di loro indicata con il simbolo `,`. Il simbolo `:-` indica la separazione tra testa (parte sinistra) e corpo (parte destra) della regola.

Sintassi: Siano `T(X,Y,K)`, `P1(M,Z,O)`, ..., `Pk(L,G,V)` atomi, allora diciamo che `T(X,Y,K) :- P1(M,Z,O), P2(X,K), ..., Pk(L,G,V)` è una regola.

Es. `Criminale(Z) :- Americano(X), Arma(Y), Vende(X,Y, Z), Ostile(Z)`

- Programma - un programma è un insieme di regole.

La tecnica del Magic Set è nata in questo ambiente allo scopo di riscrivere delle query, quindi, ottimizzare le prestazioni. Ben presto Datalog si è rivelato insufficiente per descrivere l'informazione del mondo in maniera esaustiva e nello stesso tempo compatta, essendo un linguaggio privo di funzioni. Per questo motivo passeremo ad osservare i vantaggi del Magic Set nella logica del primo ordine che si rivela estremamente vicina a Datalog.

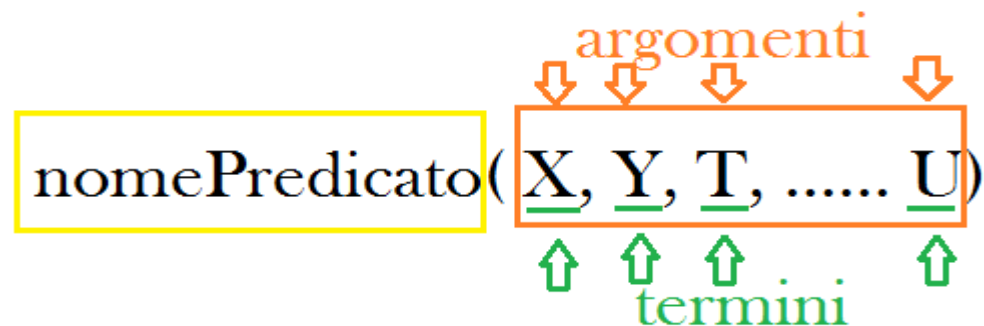
2. La logica del primo ordine

La struttura di DATALOG si può vedere come un frammento della logica del primo ordine, quindi ricapitoliamo la struttura di tale frammento a cui faremo riferimento per il resto della lettura:

- **Simboli di predicato** - rappresentano concetti del mondo reale che vogliamo modellare. Un predicato opera su più argomenti che definiscono la sua arità e può assumere due valori Vero o Falso.

Es. $Re/1$ è un simbolo di predicato unario che indica un re.

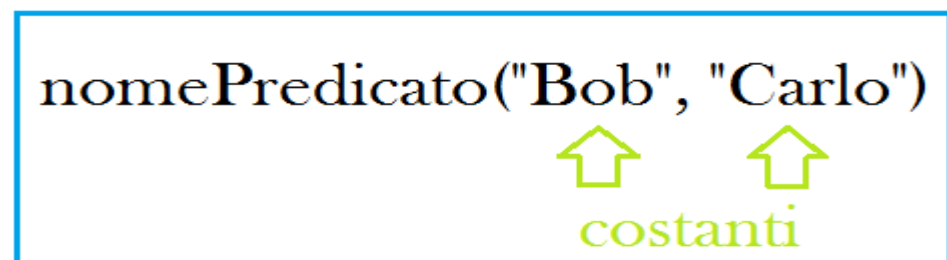
- **Atomo** - Un atomo è un simbolo di predicato applicato a termini.



Possiamo distinguere due categorie di atomi:

- **atomo ground** - Ovvero fatti noti. Questi sono simboli di predicato applicati a termini che hanno valori costanti.

atomo ground



Es. $Re(Giovanni)$ è un termine ground perché indica che la persona Giovanni è un re.

- **atomo non ground** - L'unica differenza è che questi predicati hanno come argomenti termini che sono variabili e quindi possono assumere un qualsiasi valore della nostra base di conoscenza (KB).

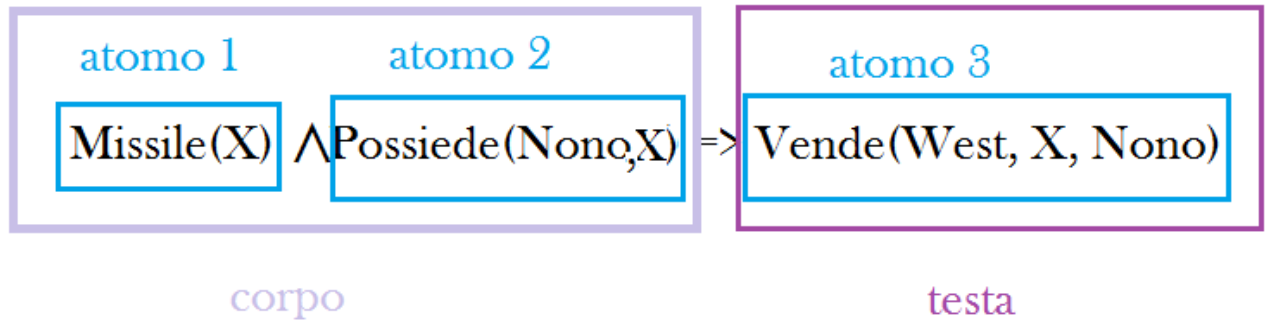
nomePredicato(X, Y, T)
↑ ↑ ↑
variabili

- **Formula** - Detta anche regola, questa esprime una caratteristica del mini mondo.
Lavoriamo con un tipo particolare di regole, chiamate regole Horn che ci permetteranno di fare inferenza logica in tempo polinomiale.

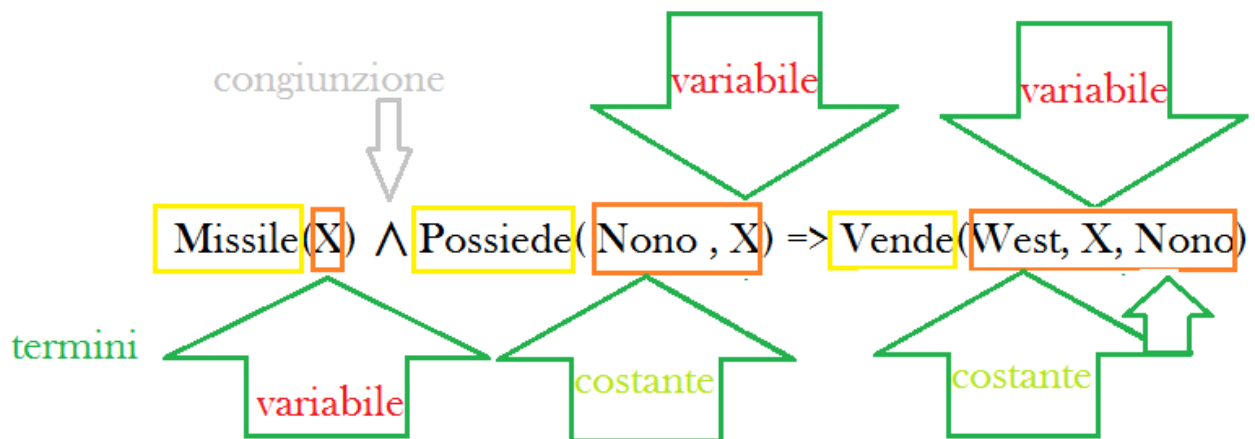
2.1 Regole Horn

Definiamo la terminologia della struttura di una regola Horn come segue:

Ogni regola è formata da un corpo ed una testa. La testa è sempre composta da un solo atomo. Il corpo è una congiunzione di zero o più atomi.



Ogni atomo è un **simbolo di predicato** che opera su **argomenti** che sono **termini** e che possono assumere valori **costanti** oppure **variabili**.



Vi ricordate le regole in Datalog?

Criminale(Z) :- Americano(X) , Arma(Y) , Vende (X,Y, Z) , Ostile(Z)

Questa regola ci dice che se una nazione è ostile e le si vendono armi da parte da un cittadino americano allora (:-) il cittadino è un criminale. Nella logica del primo ordine una struttura di questo tipo si dice regola Horn, che ha tutti gli atomi del corpo in congiunzione ed un solo atomo in testa.

Datalog: Criminale(Z) :- Americano(X) , Arma(Y) , Vende(X,Y, Z) , Ostile(Z)

FOL: Criminale(Z) $\Leftarrow$ Americano(X) $\wedge$ Arma(Y) $\wedge$ Vende(X,Y, Z) $\wedge$ Ostile(Z)

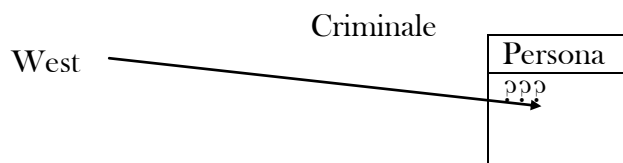
Esistono metodi d'inferenza su basi di conoscenza di tipo Horn, quali Forward-Chaining (FC) e Backward-Chaining (BC) che godono di tempi d'esecuzione di gran lunga più brevi di qualsiasi altro metodo. La soluzione dei difetti di questi metodi è l'idea movente del Magic Set. Tutto grazie alla struttura interna delle regole.

Avendo presentato come rendere la sintassi di Datalog equivalente alla sintassi di un sottoinsieme ristretto della logica del primo ordine possiamo entrare completamente nell'ambito di FOL, dove abbiamo due metodi FC e BC, essi, hanno la particolarità di permettere lo svolgimento di inferenze logiche in tempo polinomiale usando basi di conoscenza (KB) di tipo Horn. Una KB è vista come un insieme di fatti, uno per ogni tupla della corrispondente tabella del database relazionale, dove il nome della relazione diventa il simbolo del predicato del fatto. [3]

Possiede(Nono,M1) Missile(M1) Americano(West) Nemico(Nono,America)

Possiede		Missile	Americano	Nemico
Paese	Arma	Tipo Missile	Persona	Persona
Nono	M1	M1	West	West

Adesso abbiamo una visione più chiara di cosa succede dietro le quinte quando ci sottoponiamo ad un quesito, prendiamo ad esempio la query $Q = \text{Criminale}(\text{West})$. Stiamo difatti chiedendo se West è un criminale, ciò equivale a controllare l'appartenenza dell'elemento West nella tabella/relazione di nome Criminale.



Come è fatta la tabella di Criminale?

All'inizio si suppone che è vuota a differenza delle tabelle che contengono i fatti noti. Questa differenza ci fa notare una divisione chiave per tenere separati logicamente i predicati appartenenti alla base di conoscenza (EDB), da quelli che invece sono ricavati tramite deduzione logica (IDB). Lasciamo il mondo della criminalità organizzata per passare ad una realtà più semplice e vicina al lettore. Ci spostiamo nel mondo delle relazioni parentali dove definiamo i seguenti simboli di predicato (l'abuso di notazione che è introdotto dalla presenza degli attributi è necessario alla migliore comprensione del senso che portano i predicati logici):

Father(X,Y)	binario che indica che X è padre di Y.
Daughter(X,Y,Z)	ternario che indica che X è figlia di Y e Z.
Female(X) e Male(X)	d'arità uno che indicano rispettivamente che X è una femmina od un maschio.
Child(X,Y,Z)	ternario che significa che X è figlio/figlia di Y e Z.
Son(X,Y,Z)	ternario che indica che X è figlio di Y e Z.
Mother(X,Y)	binario che significa che X è madre di Y.
Parent(Y,X)	binario che indica che Y è genitore di X.

In seguito faremo riferimento a questi simboli di predicato per spiegare al meglio la divisione che esiste tra i predicati in una base di conoscenza.

2.2 EDB Extensional Database

EDB è un sottoinsieme di KB ed è formato da atomi ground. Questi racchiudono tutta la conoscenza base, cioè i fatti noti, che abbiamo a disposizione.

Es. Immaginiamo di avere una base di conoscenza composta da soli 2 maschi e 2 femmine di nome rispettivamente Philip, Charles, Elizabeth e Anne:

{ male(Philip), female(Elizabeth), male(Charles), female(Anne) }

2.3 IDB Intentional Database

Sono gli atomi non ground le cui tabelle si considerano vuote inizialmente, anche se definite per mezzo di regole precise del mini mondo, in quanto non sono ancora state popolate. Vogliamo modellarli per arrivare al risultato che vogliamo inferire. Sono noti soltanto la/le regola/e che determinano sotto che condizioni uno o più elementi della base di conoscenza possono popolare la suddetta tabella.

Es. Prendiamo l'esempio di una possibile query $\text{daughter}(X,Y,Z)$. Questo è un atomo IDB in quanto conterrà il risultato della domanda “È vero che X è figlia di Y e Z ?” Supponiamo inoltre che la regola che indica le condizioni su di esso sia:

$$\text{female}(X) \wedge \text{child}(X,Y,Z) \Rightarrow \text{daughter}(X,Y,Z).$$

In questo modo diciamo che X può assumere valori dalla relazione $\text{female}(X)$ e che se il join su X con $\text{child}(X,Y,Z)$ produce risultati allora gli elementi X,Y,Z provenienti dai risultati possono entrare a popolare la tabella di $\text{daughter}()$.

A questo punto ci manca un ultimo passo per introdurre e comprendere a pieno i metodi di inferenza logica, che stanno alla base dell'idea del Magic Set.

2.4 Modus Ponens

Abbiamo visto che per arrivare alla conclusione che un insieme di elementi appartiene ai risultati che stiamo cercando, dobbiamo fare un join su degli argomenti che sono in qualche modo simili. Questo strumento ci permette di fare conclusioni sulla base di similitudini tra gli atomi.

La regola del Modus Ponens ci permette di concludere che sapendo gli atomi ground $p_1' p_2' \dots p_n'$ ed avendo la regola $p_1 p_2 \dots p_n \Rightarrow q$ possiamo inferire q se troviamo una sostituzione Θ tale da associare un p_i' ad ogni p_i .

$$\frac{p_1' p_2' \dots p_n' \quad (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{(q)\Theta} \quad \text{dove } \Theta = (p_i', p_i).$$

Esempio: Prendiamo la regola presentata all'inizio che dice che se un re è avido allora è malvagio $Re(X) \wedge Avido(Y) \Rightarrow Malvagio(X)$. Inoltre, supponiamo di sapere che Giovanni è un re. Possiamo concludere che Giovanni è malvagio unificando X con Giovanni e K con Y :

$$\frac{Re(Giovanni), Avido(K) \quad (Re(X) \wedge Avido(Y) \Rightarrow Malvagio(X))}{Malvagio(Giovanni)} \quad \text{con } \Theta = \{X/Giovanni, K/Y\}$$

Adesso sappiamo come è costruita la logica del primo ordine, sappiamo cosa rappresentano gli atomi in una formula, come e in che categorie si dividono e come usarli per costruire regole allo scopo di modellare un mini mondo. Il prossimo passo è fare inferenza. Vediamo come fare deduzioni logiche in modo naïve ed analizzando gli svantaggi che si presentano scopriremo una tecnica che possiamo definire magica a tutti gli effetti.

3. Concatenazione all'indietro

L'algoritmo della concatenazione all'indietro prende in input una base di conoscenza di tipo Horn, un insieme goals (obbiettivi) ed un insieme Θ , inizialmente vuoto, in cui si segnerà tutte le sostituzioni per trovare l'obiettivo.

Il metodo usa la ricerca guidata dall'obiettivo, si forma un insieme di supporto in cui si inseriscono i goals da dimostrare. Si procede per unificazioni successive e si cercano per ogni atomo in goals, che si mette in q' , una regola la cui testa, singolo atomo, unifica con il atomo goal. Se si trova la sostituzione con successo si aggiungono gli atomi del corpo della regola unificata all'insieme di goals da verificare. La procedura si ripete finché l'insieme dei goals non si svuota. Questa situazione significa che abbiamo scomposto gli obiettivi nei loro sotto-obiettivi e abbiamo ripetuto fino a ridurli ad atomi ground. Una volta trasformati si procede alla verifica dell'appartenenza di tali termini ground alla nostra KB. Si ha esito positivo se essi appartengono alla base di conoscenza, altrimenti esito negativo.

Questa procedura segue un ordine sempre diverso da cui potrebbe, quasi sempre, dipendere il tempo necessario per la soluzione, infatti vedremo come un semplice controllo/ordinamento iniziale fa diventare questo svantaggio un punto di forza (vedere binding pattern) del Magic Set.

```
function FOL-BC-ASK(KB, goals,  $\theta$ ) returns a set of substitutions
  inputs: KB, a knowledge base
           goals, a list of conjuncts forming a query
            $\theta$ , the current substitution, initially the empty substitution { }
  local variables: ans, a set of substitutions, initially empty

  if goals is empty then return { $\theta$ }
   $q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(\text{goals}))$ 
  for each  $r$  in KB where  $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$ 
    and  $\theta' \leftarrow \text{UNIFY}(q, q')$  succeeds
     $\text{ans} \leftarrow \text{FOL-BC-ASK}(\text{KB}, [p_1, \dots, p_n | \text{REST}(\text{goals})], \text{COMPOSE}(\theta, \theta')) \cup \text{ans}$ 
  return ans
```

Si osserva facilmente che il fattore che rende onerosa la procedura di inferenza (usando questo approccio) è una base di conoscenza fatta da numerose regole, le teste della quali unificano con la query. Questa situazione si aggrava se le regole a loro volta sono composte da molteplici predicati nel corpo. Il BC andrà ad unificare tutte le regole e questo farà esplodere il numero di sotto obiettivi da verificare, alcuni dei quali potrebbero anche ripetersi.

Esempio: Ricordiamoci il mondo delle relazioni familiari che abbiamo descritto in precedenza. Adesso aggiungeremo dei dettagli a questo mondo per poter fare inferenza in esso. Supponiamo di avere diverse regole. Esprimiamo le condizioni sotto le quali un elemento X può essere definito padre di un elemento Y. Ovvero se X è genitore di Y e se X è maschio.

1. $\text{parent}(X,Y) \wedge \text{male}(X) \Rightarrow \text{father}(X,Y)$

Diciamo che X è madre di Y se X è femmina ed è anche genitore di Y.

2. $\text{female}(X) \wedge \text{parent}(X,Y) \Rightarrow \text{mother}(X,Y)$

Definiamo anche cosa significa essere genitore, ovvero che Y e Z sono genitori se esistono due X (stiamo spiegando regole 3 e 4 insieme) tali che sono bambini di Y o Z o tutti e due.

3. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Y,X)$

4. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Z,X)$

Inoltre specifichiamo che X è figlio di Y e Z se X è maschio ed è bambino di Y e Z.

5. $\text{male}(X) \wedge \text{child}(X,Y,Z) \Rightarrow \text{son}(X,Y,Z)$

Funziona allo stesso modo per le figlie, con la differenza che si richiede che X sia una femmina.

6. $\text{female}(X) \wedge \text{child}(X,Y,Z) \Rightarrow \text{daughter}(X,Y,Z)$

Ricordiamo come si dividono i predicati in IDB ed EDB:

Predicati	
IDB	EDB
Father/2	Male/1
Mother/2	Female/1
Parent/2	Child/3
Son/3	
Daughter/3	

Ed infine completiamo le specifiche del nostro mini mondo aggiungendo i fatti/atomii ground che saranno l'unica conoscenza certa dei oggetti nel mondo.

Fatti

7. $\text{male}(\text{Philip})$

8. $\text{female}(\text{Elizabeth})$.

9. $\text{child}(\text{Charles}, \text{Philip}, \text{Elizabeth})$

10. $\text{child}(\text{Anne}, \text{Philip}, \text{Elizabeth})$

11. $\text{male}(\text{Charles})$

12. $\text{female}(\text{Anne})$.

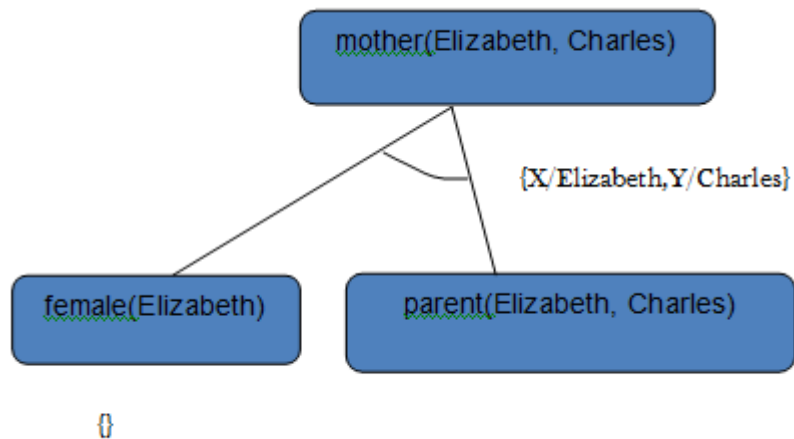
Vogliamo inferire se Elizabeth è madre di Charles.

Passo 1: si parte dall'obbiettivo

$\text{mother}(\text{Elizabeth}, \text{Charles})$

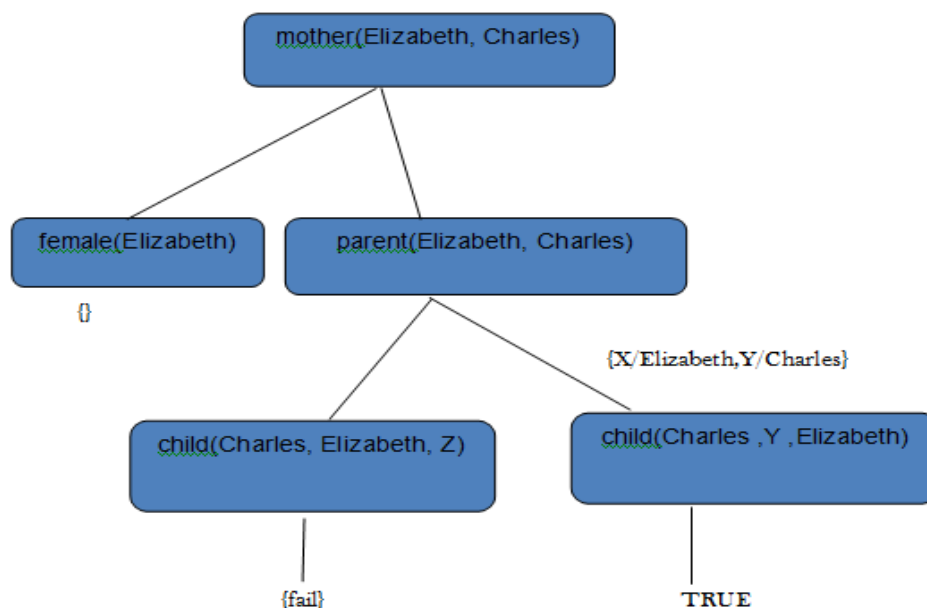
Passo 2: nel nostro insieme goals che abbiamo descritto sopra abbiamo solo l'atomo `mother(Elizabeth, Charles)` quindi procediamo alla ricerca delle regole che unificano con esso.

2. $\text{female}(X) \wedge \text{parent}(X,Y) \Rightarrow \text{mother}(X,Y)$ Questa regola ci fa aggiungere ai goals gli atomi `female(...)` e `parent(...)`. Attenzione, non dobbiamo dimenticarci l'effetto dell'unificazione che ha individuato la sostituzione $X/\text{Elizabeth}$ e $Y/\text{Charles}$ da cui i due atomi del corpo sono diventati `female(Elizabeth)` e `parent(Elizabeth, Charles)`:



Il ramo sinistro è un fatto ground quindi è soddisfatto e si passa al ramo destro.

Passo 3: Essendo `female()` soddisfatta abbiamo solo le due regole per `parent(...)` da esplorare.



Unificando con la sostituzione $(X/\text{Charles}, Z/\text{Elizabeth})$ e la regola

4. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(\text{Elizabeth}, \text{Charles})$ otteniamo `child(Charles, Y, Elizabeth)`

che appartiene all'EDB. Quindi questo ramo è soddisfatto. Così arriviamo ad inferire la nostra query.

Per sapere i motivi per cui essa è vera dobbiamo solo seguire a ritroso le unificazioni fino in cima.

Per completezza dobbiamo spiegare cosa succede alla regola 3. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Y,X)$. La sostituzione trovata con $Y/\text{Elizabeth}$ e $X/\text{Charles}$ inferisce l'atomo `child(Charles, Elizabeth, Z)` che non è presente nella KB per nessuna Z e quindi risulta in fallimento.

4. Concatenazione in avanti

Diversamente dalla BC che è guidata dall'obiettivo, questo è un metodo che usa la ricerca guidata dai dati. Prende in input una base di conoscenza con regole di tipo Horn ed una query α . Procede unificando tutte le regole della KB tra di loro e mettendo la conoscenza inferita in un insieme chiamato *new* (nel nostro caso). Controlla se la query appartiene a questa nuova conoscenza e in caso contrario ricomincia da capo ampliando KB con le conoscenze acquisite in *new* ed unificando in tutti i modi possibili.

```
function FOL-FC-Ask( $KB, \alpha$ ) returns a substitution or false
  repeat until new is empty
     $new \leftarrow \{ \}$ 
    for each sentence  $r$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$ 
      for each  $\theta$  such that  $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  is not a renaming of a sentence already in  $KB$  or new then do
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```

Ricordate l'esempio introduttivo dell'ombrello?

Adesso che abbiamo visto veramente cosa fa la concatenazione in avanti dovrebbe essere chiaro che questo approccio non è proponibile in realtà. Come fatti ground ogni persona ha la conoscenza dei numeri naturali e come ben sappiamo questi sono infiniti. Basterebbe allora saper anche sommare ed ecco che la concatenazione in avanti avrebbe cercato di inferire tutte le somme possibili nella ricerca della risposta se abbiamo bisogno di un ombrello o meno.

Esempio: Facciamo un esempio più dettagliato sul nostro mini mondo. Vediamo come si comporterebbe la FC se gli poniamo la stessa query che abbiamo posto alla BC, ovvero "Elizabeth è madre di Charles?".

Ricordiamo le regole:

1. $\text{parent}(X,Y) \wedge \text{male}(X) \Rightarrow \text{father}(X,Y)$
2. $\text{father}(X,Y) \Rightarrow \text{parent}(X,Y)$
3. $\text{father}(X,Y) \Rightarrow \text{male}(X)$
4. $\text{parent}(X,Y) \wedge \text{female}(X) \Rightarrow \text{mother}(X,Y)$
5. $\text{mother}(X,Y) \Rightarrow \text{parent}(X,Y)$
6. $\text{mother}(X,Y) \Rightarrow \text{female}(X)$
7. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Y,X)$
8. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Z,X)$
9. $\text{son}(X,Y,Z) \Rightarrow \text{child}(X,Y,Z)$
10. $\text{son}(X,Y,Z) \Rightarrow \text{male}(X)$
11. $\text{male}(X) \wedge \text{child}(X,Y,Z) \Rightarrow \text{son}(X,Y,Z)$

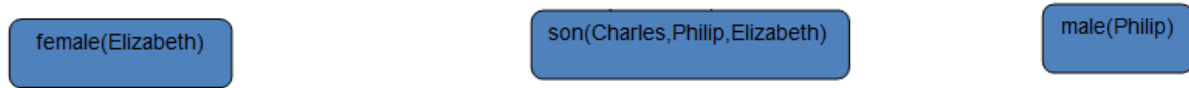
Predicati	
IDB	EDB
father/2	male/1
mother/2	female/1
parent/2	child/3
son/3	
daughter/3	

Definiamo i fatti ground:

15. $\text{male}(\text{Philip})$.
16. $\text{female}(\text{Elizabeth})$.
17. $\text{son}(\text{Charles}, \text{Philip}, \text{Elizabeth})$.

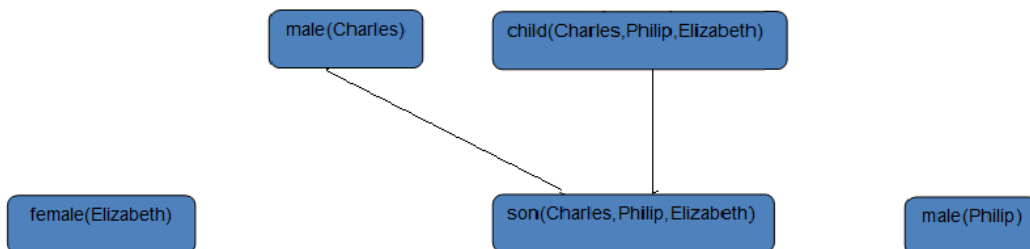
Ricordiamo anche la divisione dei predicati:

La procedura d'inferenza comincia con i tre fatti ground che abbiamo nel nostro KB.

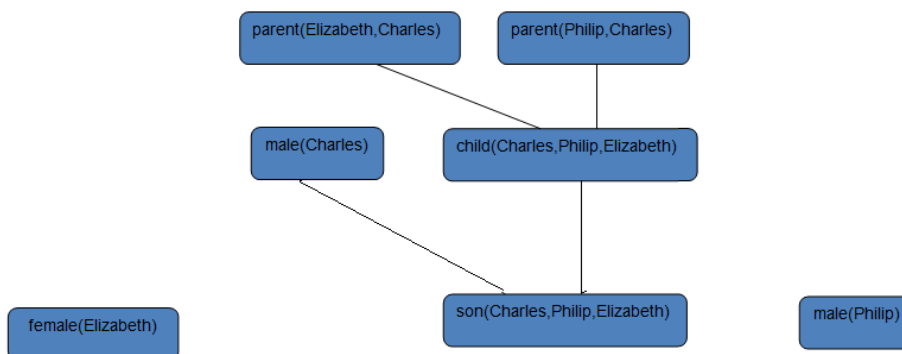


Esamina le regole e trova 9. $\text{son}(X,Y,Z) \Rightarrow \text{child}(X,Y,Z)$ dopo di che unifica $X/\text{Charles}$, Y/Philip e $Z/\text{Elizabeth}$ ed ottiene il nuovo fatto $\text{child}(\text{Charles}, \text{Philip}, \text{Elizabeth})$.

Procede alla regola 10. $\text{son}(X,Y,Z) \Rightarrow \text{male}(X)$ ed inferisce anche $\text{male}(\text{Charles})$ con la stessa sostituzione di prima.



Le nuove conoscenze che abbiamo generato ci permettono di sfruttare anche le regole che hanno come atomo nel corpo $\text{child}(X,Y,Z)$, così facendo con la sostituzione $\{ X/\text{Charles}, Y/\text{Philip}, Z/\text{Elizabeth} \}$ le due regole generano i due nuovi fatti che sono rispettivamente:
 7. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Y,X)$ si ottiene $\text{parent}(\text{Philip}, \text{Charles})$



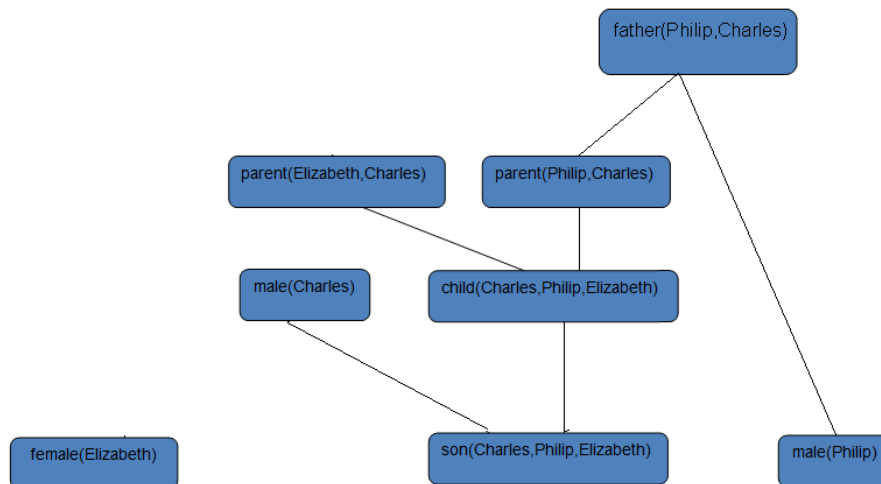
8. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Z,X)$ si ottiene $\text{parent}(\text{Elizabeth}, \text{Charles})$

Arrivati a questi fatti, siamo ad un passo dal verificare la query in quanto abbiamo a disposizione sia l'atomo $\text{parent}(\text{Elizabeth}, \text{Charles})$, predicato IDB, che il fatto ground $\text{female}(\text{Elizabeth})$. Questi due compongono il corpo della regola da cui si inferisce $\text{mother}(\text{Elizabeth}, \text{Charles})$. Sfortunatamente nell'ordine in cui le regole della KB sono esaminate la regola:

1. $\text{parent}(X,Y) \wedge \text{male}(X) \Rightarrow \text{father}(X,Y)$ precede la regola

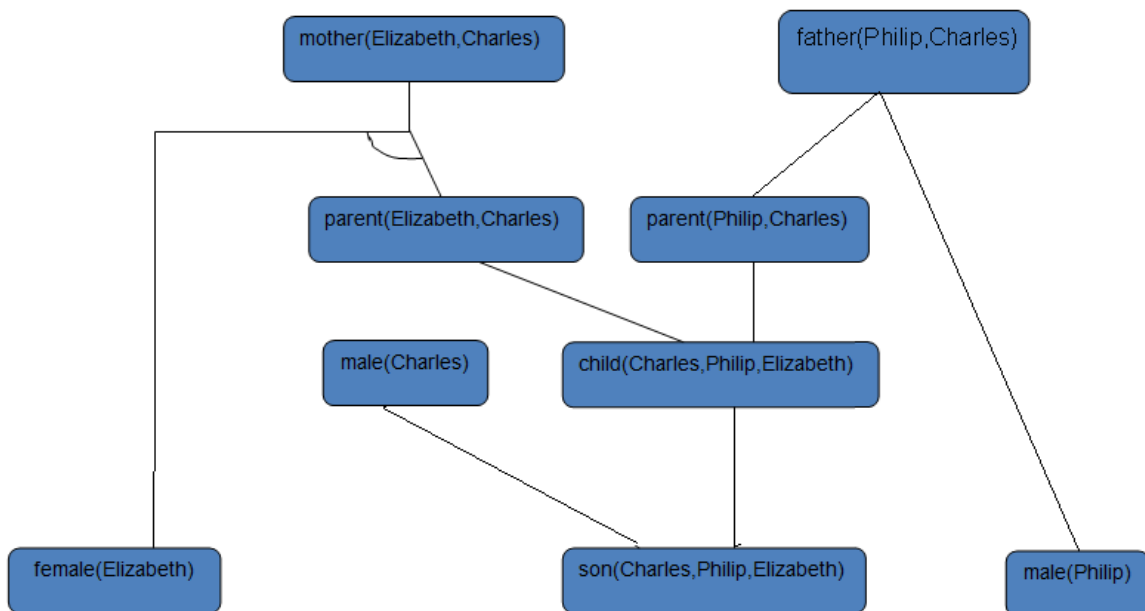
4. $\text{parent}(X,Y) \wedge \text{female}(X) \Rightarrow \text{mother}(X,Y)$,

di conseguenza FC soddisfa prima la regola numero 1 generando il fatto $\text{father}(\text{Philip}, \text{Charles})$. Ecco che osserviamo di nuovo come la mancanza di concentrazione nella logica della concatenazione in



avanti genera fatti veri ma del tutto irrilevanti.

Infine l'algoritmo giunge alla conclusione `mother(Elizabeth, Charles)` grazie alla regola numero 4 e verificando che questa è la query si conclude con successo.



Una fonte d'inefficienza nella concatenazione in avanti sembra intrinseca nello stesso approccio, infatti in ogni momento vengono fatte tutte le inferenze consentite dai fatti conosciuti. La maggior parte delle inferenze sono del tutto irrilevanti per raggiungere l'obiettivo. Nel precedente esempio questo problema salta poco all'occhio, perché ci sono poche regole che portano a conclusioni irrilevanti (vedi sottoalbero destro). In altri casi se, ad esempio, ci fossero altre regole che mescolassero le parentele con le capacità delle persone di giocare a briscola, la concatenazione in avanti genererebbe molte conclusioni inutili. Un modo per evitare questo problema è l'utilizzo delle basi di conoscenza di tipo Horn insieme all'idea di un insieme quasi magico come strumento d'aiuto per l'inferenza.

5. Magic Set

Il trucco che va sotto il nome di Magic Set consiste nel propagare vincoli aggiuntivi ai fini della restrizione dell'insieme dei fatti calcolati. Si fa ricorso ad una scansione top down per individuare il binding pattern per ogni atomo, allo scopo di definire il suo insieme magico di variabili bound (di input) e free (di output). Si passa alla trasformazione del programma (insieme di regole) nella sua versione magica. In questo modo otteniamo una versione molto più vincolata del programma di partenza, che sfrutta i vantaggi di entrambe le tecniche top down(BC) e bottom up(FC) ed evita di generare fatti non utili alla generazione della risposta. Ricordate che avere un programma sovra vincolato è sempre un vantaggio.

5.1 Binding pattern

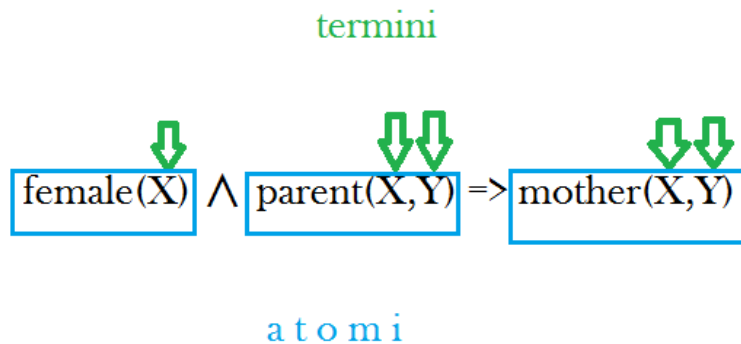
Tutto il lavoro a questo livello è svolto grazie alla concatenazione all'indietro, che parte dalla query, la quale contrassegna le regole nell'ordine conseguente-antecedente, là dove il corpo della regola da soddisfare unifica con la testa di un'altra regola nella base di conoscenza. Il primo passo verso questa limitazione consiste nell'individuare (etichettare), per ogni atomo sia IDB che EDB, quali sono i termini potenzialmente importanti. Si tende ad individuare un sovra insieme minimale dell'insieme dei simboli di predicato strettamente necessari per l'inferenza ed i loro termini importanti. Il Binding Pattern ha la particolarità di poter ridurre sensibilmente i fatti inferiti rimanendo sempre sulla strada giusta. Definiamo una serie di etichette "b" (bound) ed "f" (free) per ogni atomo sulla base dei suoi termini. Questo serve ad evidenziare il modo in cui l'informazione fluirà attraverso le regole durante il processo d'inferenza. L'ordine introdotto dalla scelta del binding pattern, in quanto collegato alla query, è diverso per ogni query e si definisce a tempo di esecuzione. Così facendo si stabilisce quali sono i termini rilevanti per ogni atomo all'interno di ogni regola, allo scopo d'inferire l'obiettivo. Seguendo solo i termini bound/rilevanti si definiscono i nuovi predicati magici.

5.1.1 L'algoritmo Binding-Pattern(KB,Q)

L'algoritmo per l'individuazione del binding pattern risolve il problema dell'assegnazione delle etichette bound e free che indicano i termini di primaria importanza per l'inferenza. Una sequenza di etichette "b" (bound) e "f" (free), che chiameremo binding pattern, si assegna ad ogni atomo. Infatti quello che si verifica spesso è che lo stesso simbolo di predicato acquisisce binding pattern diverse nelle diverse regole in cui appare come atomo. La sequenza di simboli "b" e "f" per il singolo atomo contiene un'etichetta per ogni suo termine. Inoltre all'interno di una regola ogni atomo che ha acquisito un binding pattern tramanda lo stato di "b" o "f" dei suoi termini ai successivi atomi in cui occorrono gli stessi termini. Per fare questo si seguono tre regole fondamentali che illustreremo nell'algoritmo, così si stabiliscono canali di comunicazione tra la query ed i singoli atomi, tra gli atomi stessi e tra le regole. Vedremo come questi canali diventano poi le arterie principali che alimentano con l'informazione giusta il cuore dell'algoritmo del Magic Set.

Poiché l'algoritmo ha costantemente bisogno di propagare lo stato di un termine alle successive occorrenze dello stesso nei successivi atomi della regola, cominciamo con la funzione `propagaBound(X,W)`.

Function `propagaBound(X,W)` prende in input un termine X ed un insieme di atomi W . Il suo lavoro è di segnare tutte le occorrenze di X negli atomi contenuti in W come bound. Ricordiamo che una regola è fatta come segue:



In questa figura abbiamo due occorrenze del termine X della testa negli atomi (diversi) del corpo ed una sola per il termine Y . Inoltre abbiamo una occorrenza del termine X del predicato $\text{parent}(X,Y)$ nei successivi atomi della testa (consideriamo una ordinazione da sinistra verso destra) e zero occorrenze del termine Y dello stesso atomo nei successivi atomi del corpo.

Ad esempio se chiamiamo `propagaBound()` su X e l'insieme $W = \{\text{parent}(X,Y), \text{female}(X)\}$ seguendo la figura otterremo che tutte le occorrenze del termine X nei due atomi (che formano il corpo della regola) sarebbero segnati come bound.

L'algoritmo principe del Magic Set, Binding-Pattern prende in input una KB di tipo Horn ed una query Q e fornisce in output una A_KB . A sta per Adorned (termine inglese che si traduce come adornare, decorare) la quale è una base di conoscenza che contiene un sottoinsieme (non necessariamente improprio) delle regole della KB iniziale tutte etichettate con un binding pattern per ogni atomo.

Function `Binding-Pattern(KB,Q)` Input: KB - Insieme di regole horn, Q - query

0. $A_KB = \emptyset$
1. $L = Q$ // tutti gli atomi della query
2. while $L \neq \emptyset$
3. $T(t_1, t_2, \dots, t_k) = \text{Dequeue}(L)$
4. $L2 =$ tutte le regole di KB la cui testa unifica con T
5. while $L2 \neq \emptyset$
6. $R = \text{Dequeue}(L2)$
7. $\text{AdornHead}(R, T)$ //adorna la testa
8. $\text{AdornBody}(R, L)$ //adorna il corpo
9. $A_KB + \{R\}$ //aggiungi la regola adornata all'insieme A_KB
10. return A_KB

Si fa uso di due liste di appoggio una esterna, **L**, in cui si inserisce la query e che conterrà uno o più atomi da adornare provenienti dalle regole che esamineremo in modalità top down (BC). Ed una interna, **L2**, che conterrà per ogni atomo in esame, preso dalla prima lista in riga 3, tutte le regole di KB le cui teste unificano con il predicato. In riga 6 estraiamo la prima regola di **L2** la cui testa sappiamo che unifica con **T** la variabile che contiene l'atomo goal ad ogni iterazione e poi chiamiamo le due funzioni che faranno l'adornamento della testa e del corpo della regola. Infine aggiungiamo la regola adornata al **A_KB** dove "A" sta per ADORNED. Questo procedimento si ripete per tutti gli atomi goals e subgoals e per tutte le regole in cui ognuno di loro unifica con l'atomo di testa.

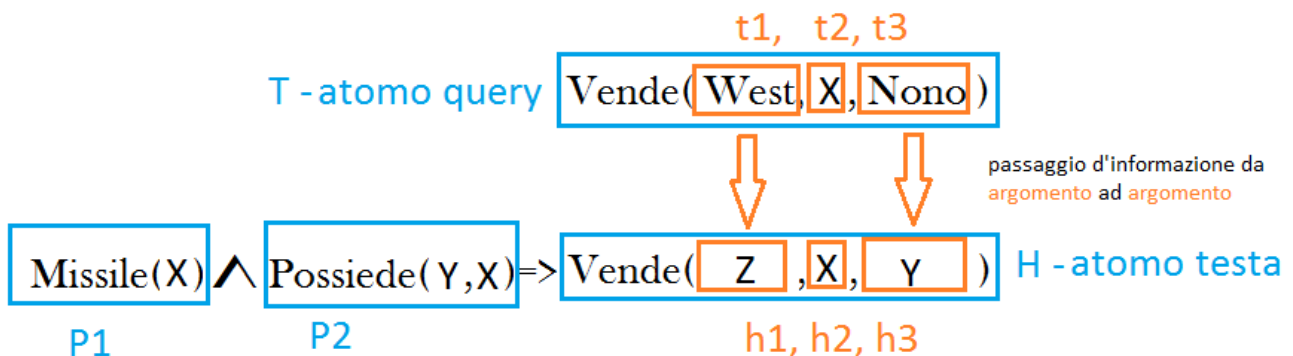
Il concetto movente della prima funzione **AdornHead** è semplice, in una query di solito si hanno poche costanti/certezze ed è strettamente necessario segnare gli argomenti che corrispondono a queste certezze come bound/importanti per tutta l'inferenza.

AdornHead($R = \{P_1(..) \wedge P_2(..) \wedge ... \wedge P_5(..) \Rightarrow H(h_1, h_2, ..., h_k)\}$, $T(t_1, t_2, ..., t_k)$)

0. for each h_j di $H(h_1, h_2, ..., h_k)$
1. if t_j è termine costante in $T(t_1, t_2, ..., t_k)$ then etichettare h_j come bound
2. else etichettare h_j come free
3. propagaBound(h_j , $\{P_1(..) P_2(..), ..., P_5(..)\}$)

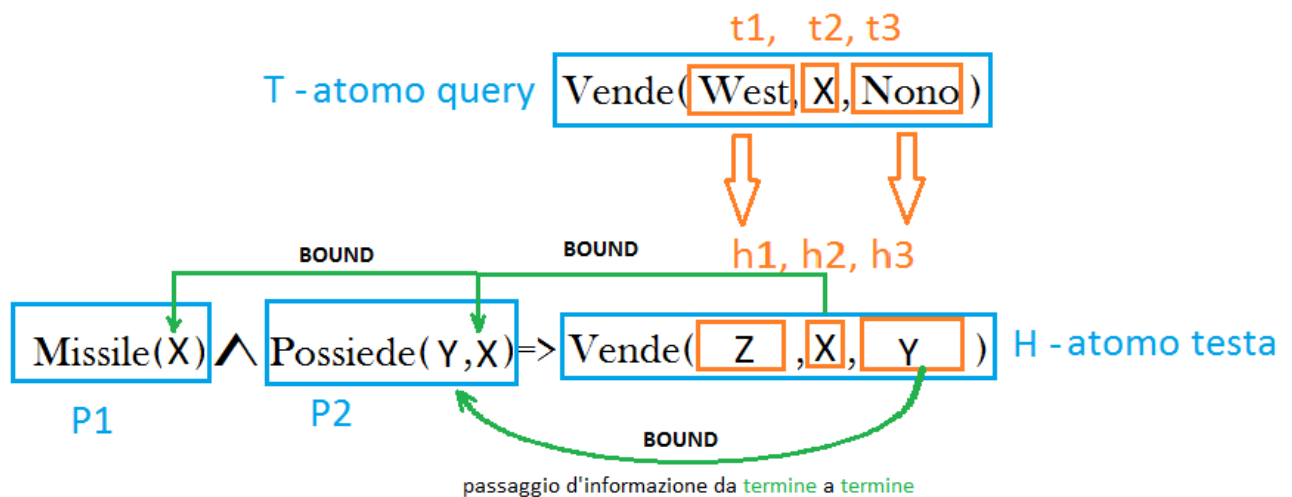
La funzione prende in input una regola **R** ed un atomo **T** ed il suo scopo ultimo è di stabilire la dipendenza tra gli argomenti della query che sono termini costanti e gli argomenti dell'atomo di testa **H** della regola **R**.

Ad esempio prendiamo la regola che dice che se **X** è un missile ed il paese **Y** possiede il missile **X** allora questo gli è stato venduto dalla persona **Z**. Qui vediamo come l'informazione fluisce per argomenti, cioè dalla costante situata in una posizione/argomento della query nella corrispettiva posizione/argomento dell'atomo di testa.



Gli atomi possono anche avere argomenti che non sono termini costanti ma termini variabili ed in tal caso non si ha nessuna informazione da passare questo è il controllo che si esegue alla riga 1.

Dopo aver etichettato tutti i termini di **H** come bound (la dove il loro corrispettivo argomento nella query è termine costante) altrimenti free, l'algoritmo assegna bound ad ogni occorrenza nel corpo di tutti i termini presenti nella testa, usando la funzione **propagaBound** (riga 3).



Il lettore potrebbe osservare che in questo modo stiamo assegnando bound a termini nel corpo che sono free nella testa ed avrebbe perfettamente ragione. Il motivo per cui questo accade è l'importanza che acquisiscono i termini con ogni passo, questa varia nei diversi punti dell'algoritmo. Se ad un certo punto un termine è free questo significa che tale termine è irrilevante fin qui, però una volta esaminato l'atomo in cui esso si incontra per la prima volta il termine acquisisce importanza per il resto dell'inferenza, cioè passa da termine di input a termine di output.

Il terzo ed ultimo adornamento che segna il un passaggio d'informazione è tra gli atomi del corpo della stessa regola. Infatti, quello che succede nei due cicli annidati in riga 0 e 1 è che si esaminano tutti i termini del corpo della regola R in un ordine prefissato. Ogni atomo comunica agli altri atomi del corpo che lo seguono, che tutti i suoi termini anche quelli free, diventano bound per tutte le loro occorrenze (riga 2, funzione `propagaBound`).

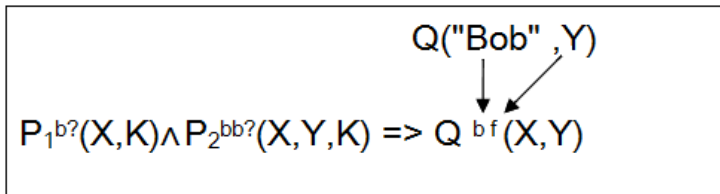
AdomBody($R = \{P_1(p_{11}, p_{12}, \dots) \wedge P_2(p_{21}, p_{22}, \dots) \wedge \dots \wedge P_s(p_{s1}, p_{s2}, \dots) \Rightarrow H(h_1, h_2, \dots, h_k)\}, L$)

0. for each P_i in R
1. for each p_{ij} in $P_i(p_{i1}, p_{i2}, \dots, p_{iz})$
2. `propagaBound`(p_{ij} , { $P_{i+1}()$, ..., $P_s()$ })
3. if p_{ij} non è bound then etichettare p_{ij} come free
4. if P_i è IDB then `Enqueue`(L , P_i)

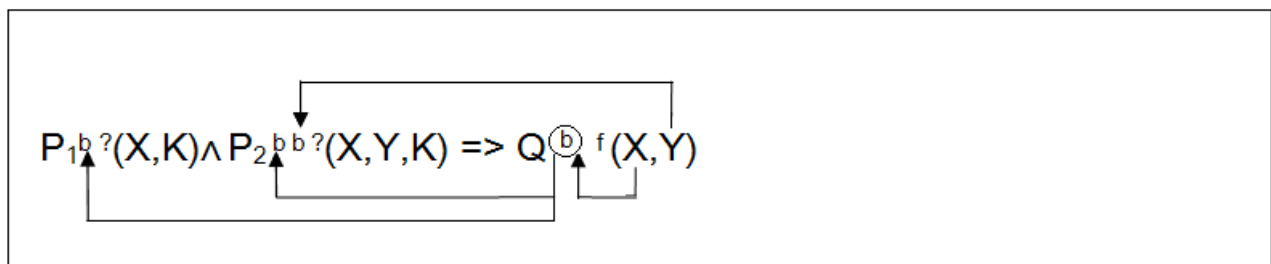
Dopo che abbiamo propagato l'importanza degli attributi nei successivi atomi si finisce l'assegnazione del binding pattern per i termini che fino a questo momento non avevano un'etichetta. Tali termini si segnano come free in riga 3. L'esaminazione di ogni singolo atomo del corpo include anche il controllo dell'appartenenza dell'atomo ad EDB o IDB (riga 4) e nel secondo caso quest'ultimo si aggiunge alla lista L . Il motivo dovrebbe essere ben chiaro a questo punto, perché le teste delle regole come abbiamo detto sono IDB e quindi dobbiamo esaminare ogni possibile regola di cui questo atomo potrebbe essere testa. D'altronde questo è anche il modo con cui BC prosegue dagli atomi di una regola alle regole in cui quest'ultimi "sono teste" (unificano con la relativa testa), la nostra scansione segue fedelmente questo approccio.

5.1.2 Esempio

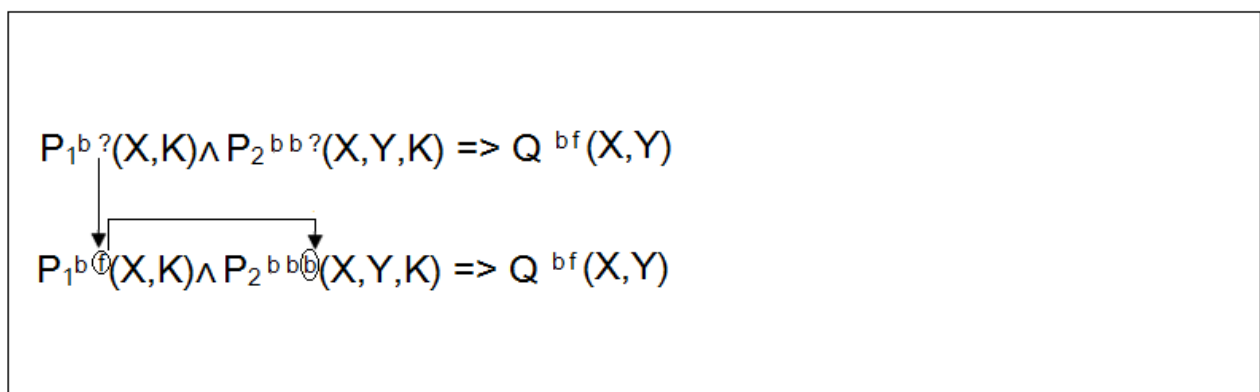
Dato l'obiettivo $Q(\text{"Bob"}, Y)$ e la regola $P_1(X, K), P_2(X, Y, K) \Rightarrow Q(X, Y)$ AdornHead dice che tutto ciò che è costante nell'atomo in esame è bound, tutto ciò che è presente nella testa è bound nel corpo. Da cui segue che X diventa bound nella testa perché nella Query (quando questa è l'atomo in esame) il suo valore è costante ovvero è "Bob" ed il termine Y rimane free nella testa per il motivo opposto.



Le occorrenze di tutti e due negli atomi del corpo però assumono lo stato di bound in quanto X e Y sono termini della testa della regola.



La seconda fase dell'algoritmo esamina gli atomi del corpo e quindi AdornBody comincia dall'atomo P_1 ed assegna free al termine K , perché questo non ha occorrenze nella testa né negli atomi che lo precedono (si presume sempre un ordine da sinistra verso degli atomi del corpo). L'algoritmo assegnerà poi a tutte le successive occorrenze del termine K lo stato di bound, in particolare questo è il termine K appartenente all'atomo P_2 .



5.2 Insiemi Magici

Dopo aver definito il binding pattern per tutti i predicati passiamo alla definizione dei loro insiemi magici. Questi sono insiemi che cominciano con una "m" (magic) e portano lo stesso nome del predicato originale a cui si appende il binding pattern dell'atomo (relativo ai termini dell'atomo originale) che ha provocato la sua creazione.

Def. Per ogni atomo $P(p_1, p_2, p_3, p_4, p_5, \dots, p_n)$ di arit  n
dati i suoi termini bound $\{p_1, p_3\}$ e termini free $\{p_2, p_4, p_5, \dots, p_n\}$ ed il binding pattern relativo a tali termini b,f,b,f si definisce il suo insieme magico $m_P_bfbf\dots f(p_1, p_3)$.

N.B. Il binding pattern che si appende al nome di un insieme magico NON si riferisce agli attributi dell'insieme magico! Ebbene questo   uno strumento che permette d'incorporare l'informazione della provenienza dell'insieme magico all'interno del proprio nome e nello stesso tempo   un modo universale per attribuire nomi diversi agli insiemi magici diversi, generati dallo stesso predicato logico, che appare come atomo in situazioni diverse in una o pi  regole.

Ad esempio prendiamo la regola $P_1(X,Y) \wedge P_2(X,Z) \wedge P_1(K,Z) \Rightarrow T(X,Z)$ dove il binding pattern   definito come segue $P_1^{bf}(X,Y) \wedge P_2^{bf}(X,Z) \wedge P_1^{fb}(K,Z) \Rightarrow T^{fb}(X,Z)$. Seguendo la definizione sopra il primo atomo genera il suo insieme magico come mostrato nella figura:

$$\begin{array}{c}
 P_1^{bf}(X,Y) \wedge P_2^{bf}(X,Z) \wedge P_1^{fb}(K,Z) \Rightarrow T^{fb}(X,Z) \\
 \downarrow \quad \downarrow \quad \downarrow \\
 m_P1_bf(X)
 \end{array}$$

Vediamo anche il secondo atomo che genera il suo insieme magico:

$$\begin{array}{c}
 P_1^{bf}(X,Y) \wedge P_2^{bf}(X,Z) \wedge P_1^{fb}(K,Z) \Rightarrow T^{fb}(X,Z) \\
 \downarrow \quad \downarrow \quad \downarrow \\
 m_P1_bf(X) \quad m_P2_bf(X)
 \end{array}$$

Dopo che il terzo atomo genera il suo insieme magico possiamo notare la furbizia della sintassi del nome degli insiemi magici:

$$\begin{array}{c}
 P_1^{bf}(X,Y) \wedge P_2^{bf}(X,Z) \wedge P_1^{fb}(K,Z) \Rightarrow T^{fb}(X,Z) \\
 \downarrow \quad \downarrow \quad \downarrow \\
 \boxed{m_P1_bf(X)} \quad m_P2_bf(X) \quad \boxed{m_P1_fb(Z)}
 \end{array}$$

Notiamo subito che il primo ed il terzo insieme magico si riferiscono in realtà allo stesso simbolo di predicato che nella formula è applicato a termini diversi e quindi genera insiemi magici diversi. Questi si distinguono grazie al binding pattern diverso che viene usato. Naturalmente lì dove il binding pattern di due atomi che si riferiscono allo stesso simbolo di predicato fosse uguale, anche se gli atomi sono applicati a termini diversi i loro insiemi magici convergerebbero in un solo insieme magico evitando duplicazione di dati.

5.3 Trasformazione magica

A questo punto della procedura si creano le nuove regole che ci aiuteranno ad arrivare magicamente dritti alla soluzione. Vediamo come basandoci sul binding pattern individuato al passo precedente, possiamo trasformare le regole/formule del nostro programma in modo da migliorarli senza perdere le interconnessioni tra di loro. In seguito useremo "insieme magico" al posto del termine "magic set" per denotare i nuovi predicati magici.

5.3.1 Passo 1: Magic facts

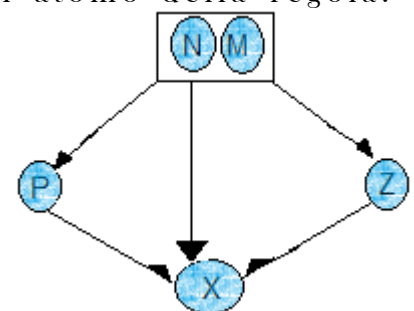
Si aggiunge la clausola unitaria, detta seme, che è l'insieme magico ricavato direttamente dalla query/obiettivo. Questo elemento servirà a due scopi quali segnare il punto d'inizio della risoluzione ed essere il rasoio più significativo tra tutti gli insiemi magici.

Ad esempio data la query `father("Bob",Y)`, possiamo facilmente dedurre il binding pattern per il predicato `father(X,Y)` che sarà X bound e Y free da cui possiamo dire che il seme sarà `m_father_bf("Bob")` dove "Bob" è costante.

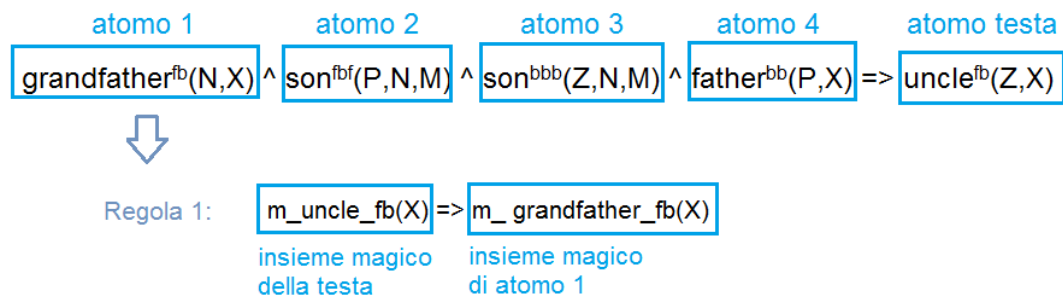
5.3.2 Passo 2: Magic rules

Sapendo come si calcola il binding pattern e come si formano gli insiemi magici per un atomo adornato, possiamo definire la trasformazione delle regole magiche che vanno aggiunte alle regole presenti nella nostra KB. Questi si calcolano a partire dagli insiemi magici degli atomi IDB presenti in ogni regola.

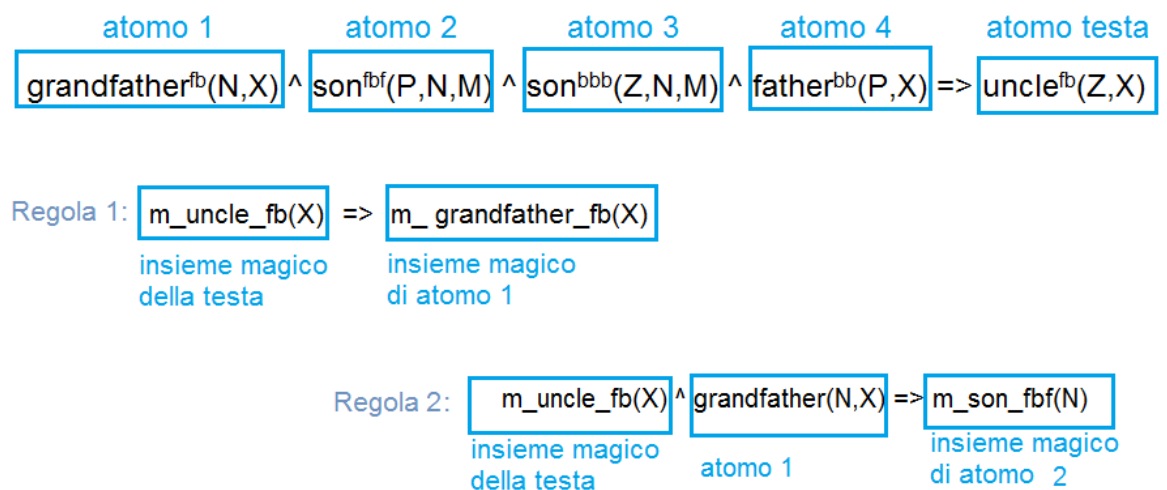
Ogni regola del KB avente k atomi nel corpo genera k regole magiche allo scopo di definire le regole secondo cui si popoleranno i nuovi insiemi magici che abbiamo creato al passo precedente, una regola per ogni insieme magico generato da ogni atomo della regola. Prendiamo ad esempio la regola che dice che se N è bisnonno di X e P è figlio della coppia N ed M ma anche Z è figlio degli stessi allora sapendo che P è padre di X segue che Z è zio di X.



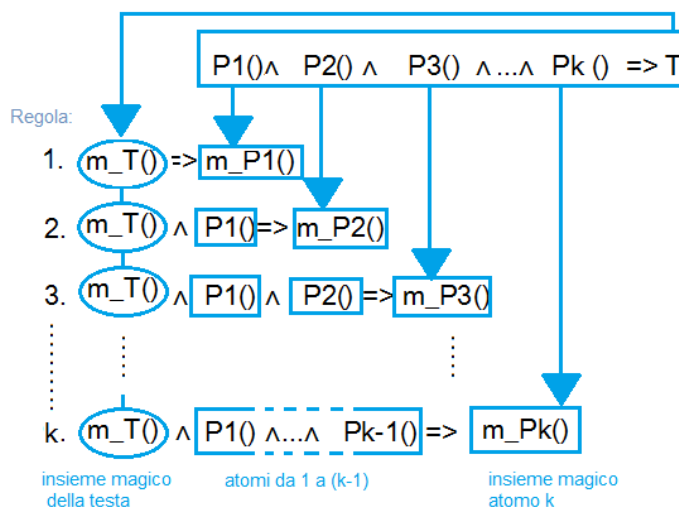
Per trasformarla si mettono in relazione per prima cosa l'insieme magico della testa e quello del primo atomo e si forma una regola:



Una seconda regola è formata dall'insieme magico del secondo predicato, che dipende dal primo predicato e dall'insieme magico della testa della regola:



E così via anche per gli atomi 3 e 4, in generale quando abbiamo una regola con k atomi la k-esima regola è generata dal k-esimo atomo mettendo in relazione l'insieme magico dell'atomo k che dipende dai primi k-1 predicati e dall'insieme magico della query.



Attenzione alcuni atomi partecipano in queste regole con la loro definizione originale !!!!! (questo sarà un punto chiave che diventerà subito chiaro nel prossimo paragrafo)

Questo passo mette in evidenza i singoli micro legami che permetteranno il passaggio d'informazione tra gli atomi del corpo, che seguono l'ordine prefissato nel binding pattern. In questo modo si crea una catena di dipendenze all'interno di ogni regola.

5.3.3 Passo 3: Modified rules

Questa trasformazione aggiunge nel corpo di ogni regola il set magico della testa come segue (per evitare qualsiasi tipo di dubbi sul binding pattern questo sarà omesso nei nomi degli insiemi magici quando si parla in generale):

Def. Data la regola $P1 \wedge P2 \wedge P3 \wedge \dots \wedge Pk \Rightarrow T$ e l'insieme magico della testa $m_T()$, la regola modificata diventa $m_T \wedge P1 \wedge P2 \wedge P3 \wedge \dots \wedge Pk \Rightarrow T$.

Ricordate l'esempio di prima dove avevamo $m_uncle_fb(Z,X)$ che restringe gli insiemi magici di tutti gli atomi del corpo della regola? Pensate ora a cosa farebbe questo passo quando sarà il momento di processare il solo atomo $IDB\ uncle(Z,X)$. Ogni predicato IDB può essere considerato regola con la sola testa ed il corpo assente. Quindi passo 1 creerà il suo insieme magico da $uncle(Z, "Carlo")$ in $m_uncle_fb("Carlo")$, passo 2 metterà in relazione questo insieme magico con tutti gli insiemi magici di tutti gli atomi in tutte le regole del KB che hanno come testa $uncle(Z,X)$ proprio come farebbe all'interno di una regola:

$m_uncle_fb(X) \Rightarrow m_grandfather_fb(X)$

$m_uncle_fb(X) \wedge grandfather(N,X) \Rightarrow m_son_fbf(N)$

$m_uncle_fb(X) \wedge grandfather(N,X) \wedge son(P,N,M) \Rightarrow m_son_bbb(Z,N,M)$

$m_uncle_fb(X) \wedge grandfather(N,X) \wedge son(P,N,M) \wedge son(Z,N,M) \Rightarrow m_father_bb(P,X)$

Infine il passo 3 salderà il legame tra tutte queste dipendenze, mettendo in relazione l'atomo con il proprio insieme magico:

$m_uncle_fb(X) \Rightarrow uncle(Z,X) !!!$

Quest'ultima regola è il cuore dell'algoritmo, in quanto senza di essa non ci sarebbe alcun collegamento tra la query e tutta la struttura di insiemi magici e legami tra le regole modificate.

Questo collegamento è la causa più comune di fallimento durante la procedura di trasformazione, in quanto è un passo trasparente all'operatore umano!!!

5.3.4. Esempio: Trasformazione Magica

Rimanendo fedeli agli esempi visti in precedenza sulle tecniche di FC e di BC trasformeremo lo stesso programma (in versione semplificata). Sottoponiamo la stessa query che vuole scoprire se Elizabeth è madre o meno di Charles.

Individuare il binding pattern

Si parte dall'obiettivo mother (Elizabeth, Charles) e come già detto si procede con la scansione in maniera BC.

$female^b(X) \wedge parent^{bb}(X,Y) \Rightarrow mother^{bb}(X,Y)$

X e Y nella testa diventano termini bound, perché sono costanti provenienti dalla query. Quindi per le regole viste sopra si propaga il loro stato di bound per ogni loro successiva occorrenza in un atomo qualsiasi della regola.

Una volta esaminata la testa, si procede con gli atomi del corpo tenendo, conto del binding pattern che ereditano dalla regola precedente. Quindi esaminiamo:

$child^{bbf}(X,Y,Z) \Rightarrow parent^{bb}(Y,X)$

dove il termine Z rimane free in quanto non appartiene alla testa, non ha ereditato lo stato di bound e non ha altre occorrenze nei successivi atomi del corpo. Si esamina anche l'altra regola la cui testa è l'atomo $parent(Z,X)$ e qui facciamo osservare l'irrelevanza dei nomi dei termini che sono attribuiti di

un atomo. I termini Z e X diventano bound grazie alla unificazione che si fa con la sostituzione che associa Y a Z ed X ad X.

$\text{child}^{\text{bfb}}(X,Y,Z) \Rightarrow \text{parent}^{\text{bb}}(Z,X)$

In questo modo Z e X ereditano per argomenti (posizioni) lo stato di bound da Y ed X. Per quanto riguarda Y vale lo stesso discorso fatto per Z nella regola precedente.

Magic facts: Seme

Una volta fissato il pattern possiamo procedere con il seme che trasforma la query `mother(Elizabeth, Charles)` in `m_mother_bb(Elizabeth, Charles)` ed aggiunge tale insieme magico al programma.

Magic Rules

Vediamo quali saranno gli insiemi magici di cui avremo bisogno. Andremo in ordine delle regole stile BC esaminando i loro atomi e gli insiemi che questi generano a partire da quelli la cui testa unifica con la query `mother(Elizabeth, Charles)`.

Per prima esaminiamo la regola

$\text{female}(X) \wedge \text{parent}(X,Y) \Rightarrow \text{mother}(X,Y)$

dove il binding pattern imposto dalla query è

$\text{female}^{\text{b}}(X) \wedge \text{parent}^{\text{bb}}(X,Y) \Rightarrow \text{mother}^{\text{bb}}(X,Y)$.

Vediamo le regole che i singoli atomi generano all'interno della stessa regola:

$\text{female}^{\text{b}}(X)$ genera $\text{m_mother_bb}(X,Y) \Rightarrow \text{m_female_b}(X)$,

$\text{parent}^{\text{bb}}(X,Y)$ genera $\text{m_mother_bb}(X,Y) \wedge \text{female}(X) \Rightarrow \text{m_parent_bb}(X,Y)$.

Il primo atomo `female(X)` è ground quindi non possiamo fare ulteriori operazioni su questo, il secondo però è `parent(X,Y)` che è un atomo IDB e quindi esaminiamo le regole con cui testa esso unifica.

La prima di queste regole (l'ordinamento delle regole con cui vengono esaminate è completamente arbitrario) è $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Z,X)$ ed il suo binding pattern tramandato da $\text{parent}^{\text{bb}}(X,Y)$ è $\text{child}^{\text{bfb}}(X,Y,Z) \Rightarrow \text{parent}^{\text{bb}}(Z,X)$.

Abbiamo un unico atomo `child(X,Y,Z)` che genera $\text{m_parent_bb}(Y,X) \Rightarrow \text{m_child_bfb}(X,Y)$.

E poi passiamo alla prossima regola

$child(X,Y,Z) \Rightarrow parent^{bb}(Y,X)$ che induce il binding pattern $child^{bbf}(X,Y,Z) \Rightarrow parent^{bb}(Y,X)$ e quindi il suo unico atomo $child(X,Y,Z)$ genera la regola $m_parent_bb(Z,X) \Rightarrow m_child_bfb(X,Z)$. Anche questo è un atomo ground e quindi il lavoro dell'algoritmo sulla generazione degli insiemi magici delle regole si ferma.

Modified Rules

Passiamo ad esaminare i legami che dobbiamo stabilire tra le vecchie regole ed i nuovi insiemi che abbiamo creato. Come già spiegato al passo 3 dobbiamo collegare l'insieme magico della testa con la relativa regola per ogni regola in KB.

Quindi la regola $female(X) \wedge parent(X,Y) \Rightarrow mother(X,Y)$ diventa $m_mother_bb(X,Y) \wedge female(X) \wedge parent(X,Y) \Rightarrow mother(X,Y)$,
la regola $child(X,Y,Z) \Rightarrow parent(Y,X)$ diventa $m_parent_bb(Y,X) \wedge child(X,Y,Z) \Rightarrow parent(Y,X)$
ed infine la regola $child(X,Y,Z) \Rightarrow parent(Z,X)$ diventa $m_parent_bb(Z,X) \wedge child(X,Y,Z) \Rightarrow parent(Z,X)$.

Facciamo osservare al lettore che la trasformazione magica si è fermata perché ha esaurito le regole che ha considerato necessari all'inferenza della query. Questo fatto illumina una caratteristica difficilmente intuibile riguardo il Magic Set. La tecnica non solo tiene concentrata l'inferenza ma restringe anche l'insieme di regole del mondo su cui l'inferenza deve agire. Pensate che allo scopo d'inferire questa esatta query $mother("Elizabeth", "Charles")$ noi possiamo immergere le 3 regole toccate dal Magic Set in un insieme di altre regole vasto a piacere e la quantità di lavoro che dovremmo fare rimarrà immutata. Senza questa proprietà non si riesce neppure a immaginare la quantità di informazione irrilevante che avrebbe prodotto l'applicazione della concatenazione in avanti.

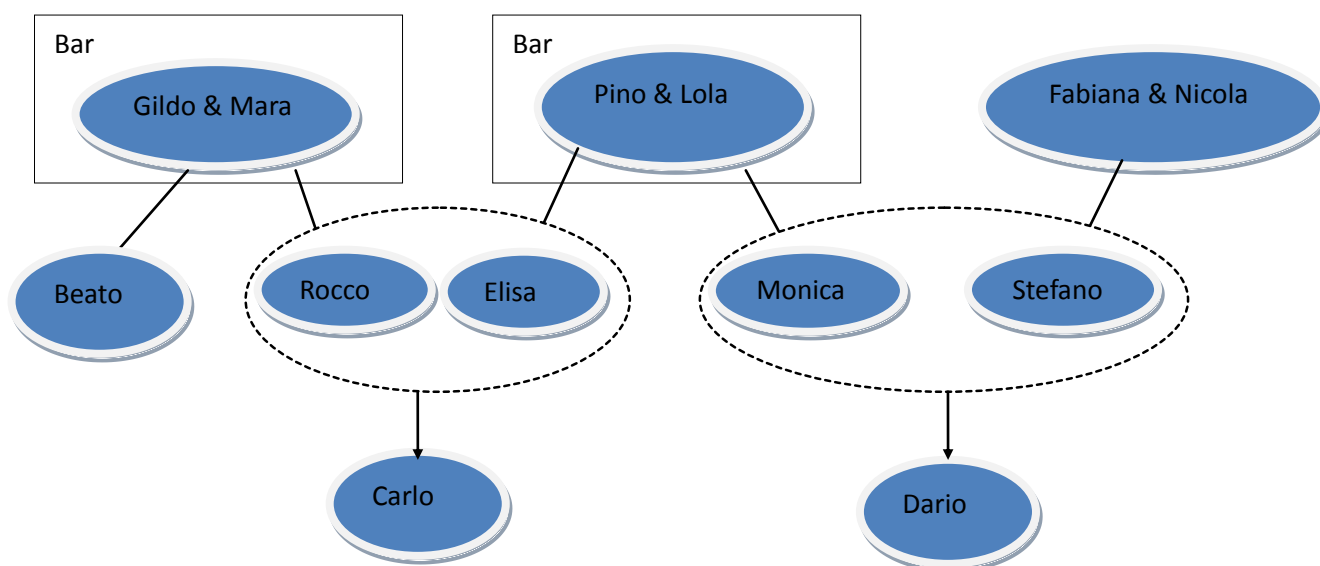
6. Conclusioni

Alla luce delle tecniche d'inferenza viste anche durante il corso, abbiamo illustrato le capacità del Magic Set, però ancora non abbiamo fatto il confronto diretto tra i tre. Con il seguente esempio facciamo vedere il tocco di classe che si introduce grazie alle limitazioni delle inferenze nel corso di una risoluzione e metteremo in atto FC BC e MS sullo stesso programma.

Prima però vogliamo evidenziare un punto molto delicato, Magic Set non è un metodo ma un miglioramento, un ibrido tra FC e BC. Quindi in termini di complessità temporale la situazione rimane tale quale - polinomiale, grazie alla struttura delle regole (horn). Il vero vantaggio viene a livello di conoscenza inferita, in quanto il Magic tiene sempre concentrata la procedura di risoluzione senza permettere deviazioni dal percorso, anche se esse sono valide e vere, ma irrilevanti. Il prezzo da pagare per usare il Magic Set c'è e, come tutti i predicati magici e strutture aggiuntive, per non parlare delle regole trasformate occupano comunque spazio. Non dobbiamo dimenticarci che la nostra base di conoscenza è formata da relazioni/tabelle, tra le quali l'informazione si ottiene per mezzo di join pesanti, che coinvolgono un numero superiore di tabelle rispetto a quello che dovrebbero coinvolgere senza la trasformazione magica.

Dario gioca a briscola? E Carlo?

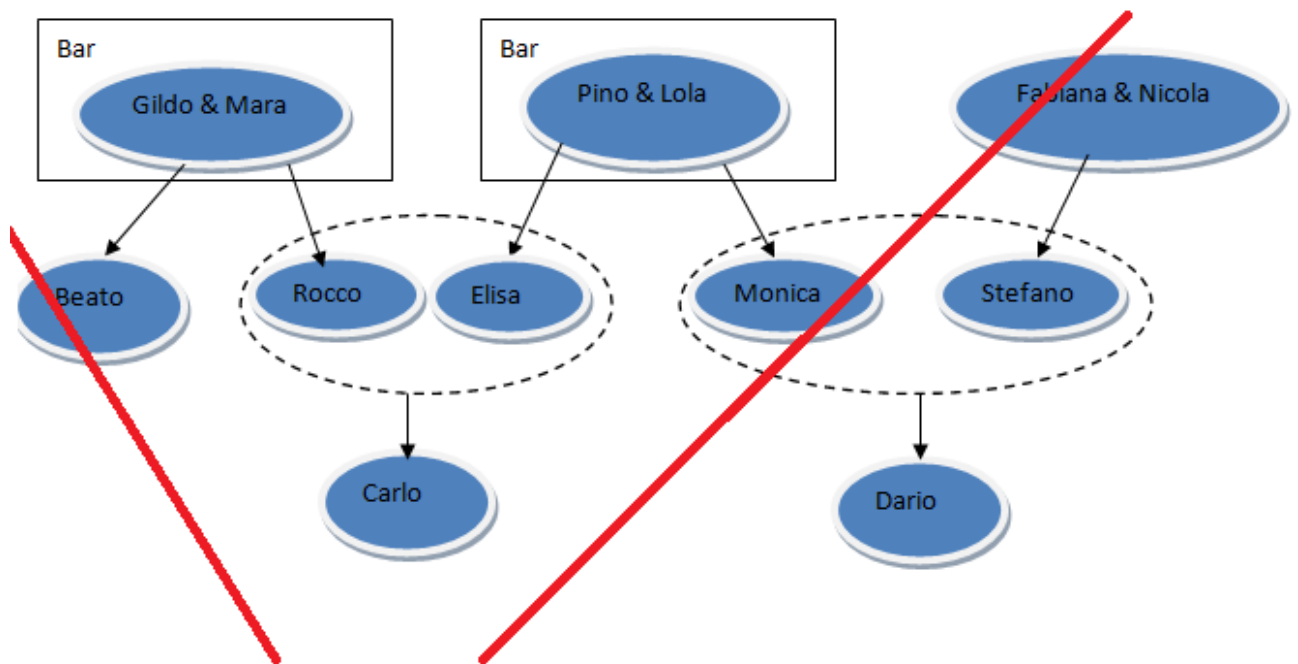
Assumiamo di essere in un mondo chiuso in cui ci sono solo le persone dalla figura sottostante. Le regole di questo mini mondo sono: se uno possiede un bar allora sa giocare a briscola, un bambino sa giocare a briscola se suo padre sa giocare a briscola e suo nonno/a ha un bar (di conseguenza il nonno/a sa giocare). Si presuppone che c'è la prassi, da parte dei genitori molto occupati con il lavoro, a lasciare spesso il proprio figlio/a da piccolo dai nonni al bar. Pertanto, capita spesso che il bambino giocando insieme al padre e ai nonni impari a giocare a briscola. In tutte le altre casistiche, se i nonni dalla parte del padre non hanno il bar, il bambino non sa giocare a briscola. Le mamme e le figlie, essendo giovani, preferiscono il flipper alla briscola diversamente dalle nonne.



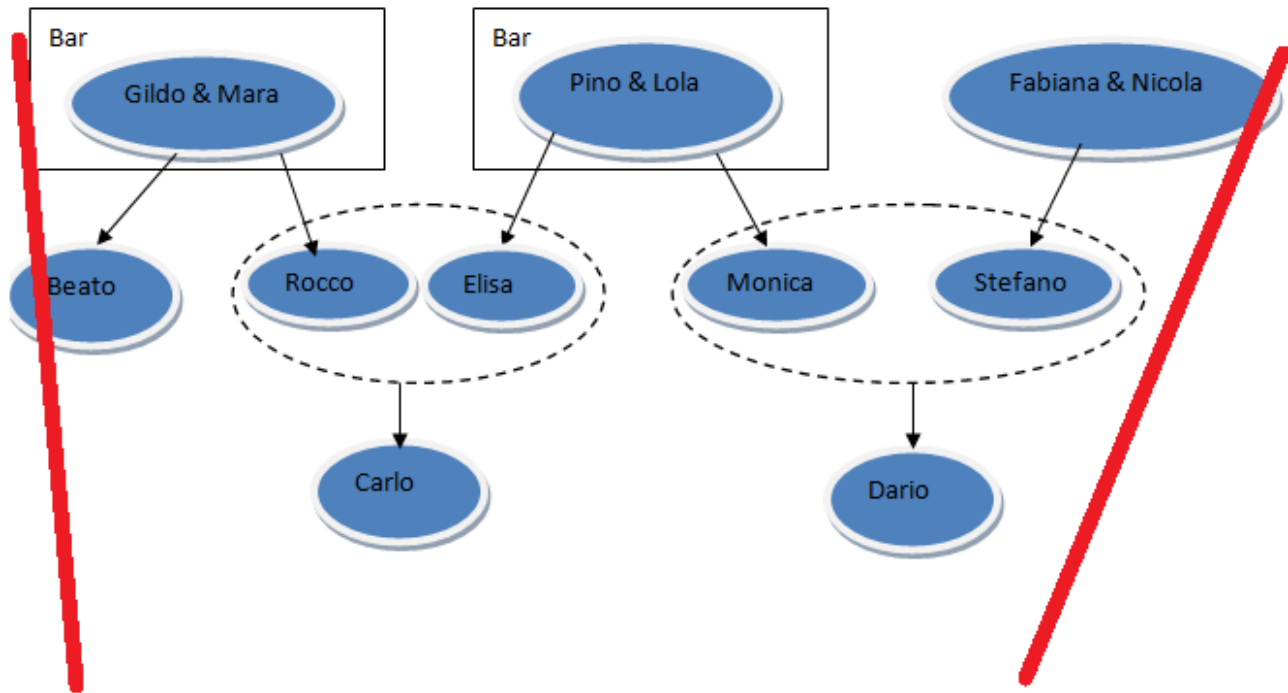
Formalizziamo questo mini mondo in regole come segue. Inoltre introduciamo un nuovo simbolo di predicato unario “gb” che indica che una persona sa giocare a briscola. I simboli di predicato grandfather/2 e grandmother/2 portano il significato che si intuisce dalla loro traduzione dall’inglese rispettivamente nonno e nonna.

1. $\text{parent}(X,Y) \wedge \text{male}(X) \Rightarrow \text{father}(X,Y)$
2. $\text{father}(X,Y) \Rightarrow \text{parent}(X,Y)$
3. $\text{father}(X,Y) \Rightarrow \text{male}(X)$
4. $\text{parent}(X,Y) \wedge \text{female}(X) \Rightarrow \text{mother}(X,Y)$
5. $\text{mother}(X,Y) \Rightarrow \text{parent}(X,Y)$
6. $\text{mother}(X,Y) \Rightarrow \text{female}(X)$
7. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Y,X)$
8. $\text{child}(X,Y,Z) \Rightarrow \text{parent}(Z,X)$
9. $\text{son}(X,Y,Z) \Rightarrow \text{child}(X,Y,Z)$
10. $\text{son}(X,Y,Z) \Rightarrow \text{male}(X)$.
11. $\text{male}(X) \wedge \text{child}(X,Y,Z) \Rightarrow \text{son}(X,Y,Z)$
12. $\text{father}(X,Z) \wedge \text{father}(Z,Y) \Rightarrow \text{grandmother}(X,Y)$
13. $\text{father}(X,Z) \wedge \text{father}(Z,Y) \Rightarrow \text{grandfather}(X,Y)$
14. $\text{nonno}(X,Y) \wedge \text{nonna}(Z,X) \wedge \text{father}(K,X) \wedge \text{gb}(Y) \wedge \text{gb}(Z) \wedge \text{gb}(K) \wedge \text{male}(X) \Rightarrow \text{gb}(X)$
15. $\text{grandmother}(K,X) \wedge \text{bar}(K) \Rightarrow \text{gb}(X)$
16. $\text{grandfather}(K,X) \wedge \text{bar}(K) \Rightarrow \text{gb}(X)$
17. $\text{father}(X,Y) \wedge \text{gb}(Y) \Rightarrow \text{gb}(X)$

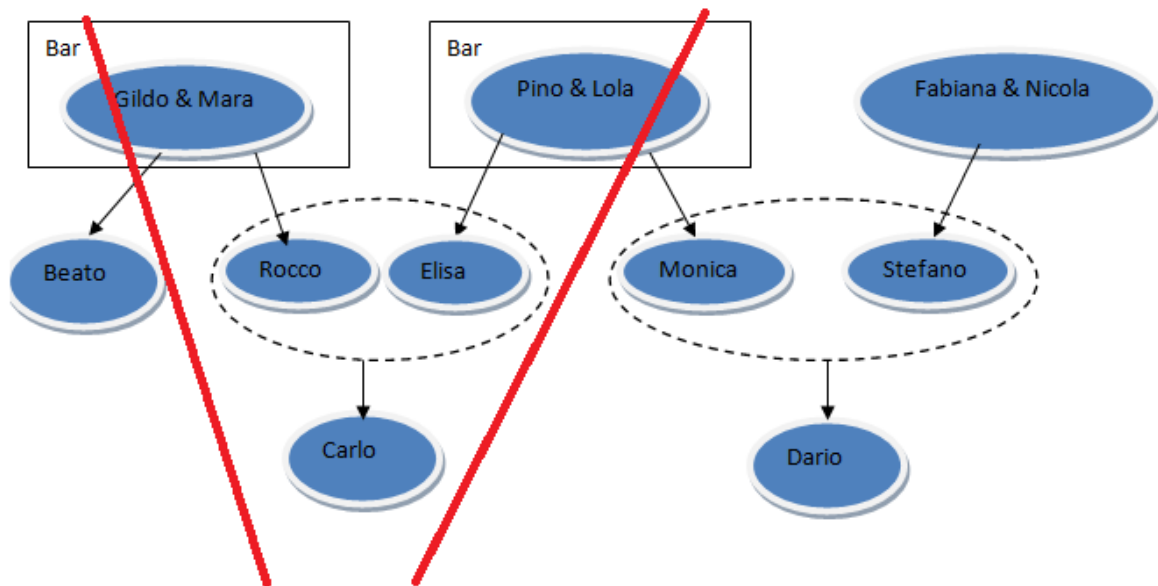
In questo mondo simulato la concatenazione all'indietro, che si trova a rispondere alla domanda se Carlo gioca a briscola, affonda. Ovvero incontrerà la regola 14 e poi dovrà inferire tutti i predicati del corpo, cioè tutti i nonni, nonne, padri, maschi e giocatori a briscola, finché non arriverà al punto fisso. La soluzione, se c'è, apparterrà di sicuro all'insieme dei fatti generati, ma l'albero dell'inferenza esploderà in ampiezza per colpa dei fatti dedotti, veri ma irrilevanti. Per esempio si giungerà alla conclusione che Beato (zio di Carlo) sa giocare, ma questa informazione non ci interessa data la query $gb(Carlo)$. Come osservato in precedenza la difficoltà qui arriva dalle regole con un corpo lungo e composto da numerosi predicati. Nel prossimo disegno diamo un'idea grossolana della quantità d'informazione che andrà ad analizzare BC, delimitata in rosso.



Neanche la concatenazione in avanti se la caverà meglio, in quanto comincerà ad inferire tutta la conoscenza possibile che può estrarre dal DB. Con "l'aiuto" di tanti termini ground riusciamo ad ostacolare anche questa procedura, nonché far inferire tutte le parentele dei personaggi e solo allora tutte le loro capacità di giocare o meno a briscola. In ultima analisi anche FC esplode in ampiezza ed inferisce tutta la conoscenza ricavabile dal modello, anche se serve molto meno per giungere alla risposta come vedremo con la prossima analisi.



La situazione è ben diversa usando Magic Set, perché quando chiediamo $gb(\text{Carlo})$, dietro le quinte la tecnica segnerà come fatti utili solo quelli predicati connessi direttamente con l'obiettivo. In questo caso saranno comunque inferiti relazioni di parentela, però saranno solo quelle legate a Carlo che ci portano, in modo contenuto, alla soluzione.



Bibliografia

- [1] "Intelligenza artificiale. Un approccio moderno", Volume 1
- [2] kdd.isti.cnr.it/~nanni/papers/tesi.ps.gz
- [*] Magic Sets Example of Nonrecursive Use Recursive Example <http://infolab.stanford.edu/~ullman/cs345notes/slides01-14.pdf>
- [*] Magic Sets and Other Strange Ways to Implement Logic Programs http://pdf.aminer.org/000/300/337/implementing_logic_programs_as_a_database_system.pdf
- [*] slides 01-15, Rule-goal Trees and "Optimal" Evaluation of Rules
- [*] slides 01-16, The Magic-Sets Algorithm
- [*] "Magic Templates: A Spellbinding Approach To Logic Programs", Raghu Ramakrishnan, The Journal Of Logic Programming, 1991
- [*] Intelligenza Artificiale, Lezione 10, Daniele Nardi, 2004
- [*] "Magic-Sets for Datalog with Existential Quantifiers", M. Alviano, N. Leone, M. Manna, G. Terracina, P. Veltri, Department of Mathematics, University of Calabria, Italy
- [*] "Deductive Database and Logic Programming", Stefan Brass, Univ. Halle, Winter 2007-2008
- [*] "Recursive SQL, Deductive Databases, Query Evaluation", Book Chapter of Ramakrishnan and Gehrke DBMS Systems, 3rd edition
- [*] "Database Management Systems", 3ed, R. Ramakrishnan and J. Gehrke, Chapter 25
- [*] "Linguaggi di query basati sulla programmazione logica", Letizia Tanca, <http://home.deib.polimi.it/schreibe/BasidiDati0607/LucidiTeoria/LinguaggioDatalog.pdf>
- [*] "Datalog", Politecnico di Milano, Dip. di Elettronica e Bioingegneria, <http://home.deib.polimi.it/campi/dbIol/Lezioni/datalog.pdf>
- [*] "Datalog", UMBC, <http://www.csee.umbc.edu/~pmundur/courses/CMSX661-05/datalog.ppt>
- [*] "Principles of database and knowledge - base systems", volume II: The New Technologies, J.D. Ullman
- [*] "Magic Sets and their Application to Data Integration", W. Faber, G. Greco, N. Leone, Dip. of Mathematics University of Calabria, Italy
- [*] "Agenti logici: sistemi a regole", Maria Simi a.a. 2008-2009
- [*] "Magic is Relevant", I. S. Mumick, S. J. Finkelstein, H. Pirahesh, R. Ramakrishnan
- [*] "Datalog and Emerging Applications: an Interactive Tutorial", S. S. Huang, T. J. Green, Boon, Athens Greece 2011
- [*] "Sistemi di ragionamento logico", <http://www.dti.unimi.it/pizzi/Allezioni/SISTEMI%20DI%20RAGIONAMENTO%20LOGICO.doc>
- [*] "Datalog Bottom-up is the Trend in the Deductive Database Evaluation Strategy", Yurek K. Hinz, University of Maryland, 2002
- [*] "Database Management Systems", 3ed, R. Ramakrishnan and J. Gehrke, Chapter 25
- [*] "Linguaggi di query basati sulla programmazione logica", Letizia Tanca, <http://home.deib.polimi.it/schreibe/BasidiDati0607/LucidiTeoria/LinguaggioDatalog.pdf>
- [*] "Datalog", Politecnico di Milano, Dip. di Elettronica e Bioingegneria, <http://home.deib.polimi.it/campi/dbIol/Lezioni/datalog.pdf>
- [*] "Datalog", UMBC, <http://www.csee.umbc.edu/~pmundur/courses/CMSX661-05/datalog.ppt>
- [*] "Principles of database and knowledge - base systems", volume II: The New Technologies, J.D. Ullman
- [*] "Magic Sets and their Application to Data Integration", W. Faber, G. Greco, N. Leone, Dip. of Mathematics University of Calabria, Italy
- [*] "Agenti logici: sistemi a regole", Maria Simi a.a. 2008-2009
- [*] "Magic is Relevant", I. S. Mumick, S. J. Finkelstein, H. Pirahesh, R. Ramakrishnan
- [*] "Datalog and Emerging Applications: an Interactive Tutorial", S. S. Huang, T. J. Green, Boon, Athens Greece 2011
- [*] "Sistemi di ragionamento logico", <http://www.dti.unimi.it/pizzi/Allezioni/SISTEMI%20DI%20RAGIONAMENTO%20LOGICO.doc>
- [*] "Datalog Bottom-up is the Trend in the Deductive Database Evaluation Strategy", Yurek K. Hinz, University of Maryland, 2002